Anti-Corruption Base Intelligence

BaseLine

Plantel: Centro Universitario de los Valles

Materia: Software Configuration Management

Nombre del profesor: *Omar Ali Zatarain Durán*

Grado: Tercer semestre

Prepared by: Hiram Gabriel Alonso Marquez

Index

Objectives **2**
See the following documents: 2
Functional Architecture **3**
Technical Implementation **3**
Software Stack & Architecture 3
Non-Functional Security Requirements 4
Development Methodology **4**

Introduction Anti-Corruption Base Intelligence (AcBi)

The "AcBi" project (Anti-Corruption Base Intelligence) is the design and development of a cross-platform mobile application for Android and iOS. It is specifically engineered for researchers, journalists, and civil society organizations to facilitate the proactive detection of corruption patterns within the public sector. The platform utilizes a local-first, offline approach to ensure data integrity and user safety in high-stakes environments.

We have worked on defining the main modules, their functional and non-functional requirements, the flow of the architectural logic, product perspective, user types, the type of contract, Development Methodology, and its design.

Objectives

The primary objective of AcBi is to provide a forensic intelligence tool capable of identifying sophisticated anomalies in public administration:

- Irregular Awards: Detection of anomalies in government tenders and contracts.
- Hidden Beneficiary Networks: Mapping complex relationships to find illicit connections.
- Time Paradoxes: Identifying chronological inconsistencies, such as approvals granted before official requests were filed.

See the following documents:

These documents form the basis of the present project.

- Software Requirements Specification (SRS) V-1
- Contract Type-V-1
- Design Overview-V-1
- Resource Planning

Functional Architecture

The solution integrates five specialized modules to manage the investigative workflow, seeTable 1. Modules:

Module	Primary Function
Data Listener	Real-time capture and monitoring of public data from configurable APIs.
Network Analysis	Construction of dynamic relationship graphs and automated risk scoring.
Process Tracking	Detailed monitoring of bureaucratic lifecycles and tender processes.
Relational Time Lines (RTL)	Temporal comparison between official declarations and captured signals.
Time Paradox	A central dashboard for alerts and the detection of chronological anomalies.

Table 1. Modules

Technical Implementation

Software Stack & Architecture

- Architecture: Built on Clean Architecture principles combined with a local-first design.
- Frontend: Developed using Tauri 2.0 Mobile for cross-platform efficiency.
- Backend: Powered by Rust for high performance and advanced code obfuscation.
- API Manager: A key innovation allowing investigators to manually configure URLs, endpoints, and parsing rules while validating them in real time via dynamic diagrams.

Non-Functional Security Requirements

The platform incorporates 20 extreme security requirements to protect sensitive investigations:

- Encryption: Comprehensive End-to-End Encryption (E2EE).
- Chain of Custody: Immutable digital logs using SHA-512 hashes and certified timestamps.
- Operational Anonymity: Integrated Tor/VPN support with automatic IP rotation.
- Counterintelligence: Protection against reverse analysis, sandbox detection, and memory dump prevention.
- Self-Protection: Features including a reverse honeypot and programmable self-destruct mechanisms to prevent unauthorized data access.

Development Methodology

The "AcBi" project is a multiplatform mobile application designed for researchers for the proactive detection of corruption. We will utilize an Agile methodology (Scrum). The system follows a Clean Architecture approach, organizing code into independent layers to facilitate maintenance:

- Development Approach It will follow a Hybrid Development model: React with JavaScript for interactive UIs like dashboards, timelines, and charts via vis.js and D3.js with Rust for logic bussiness and obfuscation.

- The system uses Tauri with React Native for a multiplatform app, PC, and Android.

- The structure is Clean Architecture with Local-First (Storage). To maintain total control over sensitive data, the app runs without third-party central servers.

- FrontendUI is made by React Native with vis.js and D3.js for the dashboard, visualizations, and dynamic diagrams.

- Backend will be made with Rust for main modules, API management using Go to make many requests at the same time.

- The Encrypted Database is local storage using SQLCipher in the test devices.

- Comprehensive testing will be conducted for unit tests for cryptography, in order to ensure anonymity and counter-intelligence protection.

- SHA family for hashing, cryptography, VPN support, and custom Rust-based anti-forensics (Obfuscator.io for code protection), sandbox, and memory dump detection to prevent the malicious agent from obtaining a copy of the contents of RAM at a specific time.